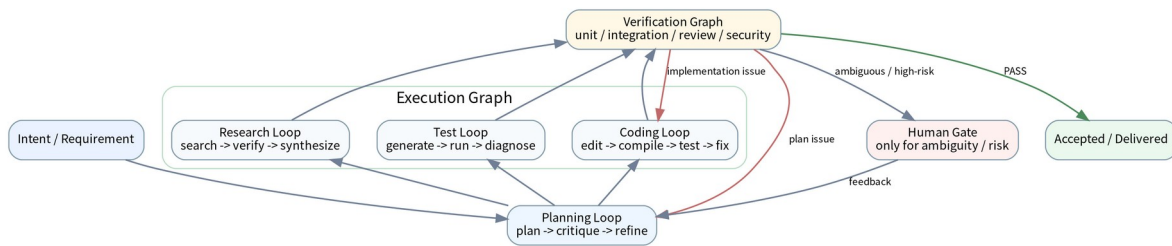


# Graph Engineer VS Loop Engineer

## AI Agent 工程从反馈循环到执行图谱的深度研究报告

核心主题: Execution Topology • Feedback Loop • Context Isolation • Durable State • Failure Routing • Graph of Loops



研究时间: 2026-08 • 基于公开框架文档、官方技术博客与工程化分析

# 0. 执行摘要

先回答“谁替代谁”之前，先把层级关系说清楚。

核心结论

Graph Engineer 不是 Loop Engineer 的简单“下一代替代品”。Loop 解决的是反馈收敛；Graph 解决的是全局编排。Loop 是 Graph 的基本结构之一，成熟生产系统通常应采用“Graph 外层 + Loop 节点内层”，即 Graph of Loops。

如果把一个 Agent 看成一个软件工程师，Loop Engineering 解决“让这个工程师持续做、持续检查、持续修正直到达标”；Graph Engineering 解决“把规划、研究、编码、测试、审核、人类审批等角色组织成一个可编程的软件团队，并明确状态、路由、并行、失败恢复和权限边界”。

## 0.1 一页结论

| 问题    | Loop Engineer                      | Graph Engineer                             |
|-------|------------------------------------|--------------------------------------------|
| 核心对象  | Feedback Loop                      | Execution Graph / Control Plane            |
| 优化目标  | Convergence：持续逼近目标                 | Coordination：复杂任务的正确分工与路由                  |
| 基本单位  | Agent iteration / verifier / retry | Node / Edge / State / Router / Subgraph    |
| 控制流   | 循环为中心                              | 顺序、条件、并行、汇合、循环、人工中断                        |
| 上下文策略 | 单会话或轮次级压缩/重启                       | 节点级 context isolation + structured handoff |
| 失败处理  | 重试整轮或反馈回 Agent                     | 局部失败隔离，按错误类型回到特定节点                         |
| 最适任务  | 开放式、单主线、长时间自我修正                    | 多阶段、多角色、高价值、可并行、强治理任务                      |
| 主要风险  | context rot、死循环、错误累积               | 编排过度、协调成本、并行写冲突、状态复杂度                      |
| 推荐关系  | 作为局部执行单元保留                         | 作为系统级外层控制结构                                |

## 0.2 最重要的判断

- 不要因为“Graph Engineering”流行就把所有 Agent 都拆成多 Agent。简单 bug 修复、单文件修改、强串行任务通常仍应优先使用 Loop。
- Graph 的价值不在“Agent 数量”，而在“工程师显式控制拓扑”：谁能做什么、何时做、拿什么上下文、失败去哪里、什么时候必须停。
- Graph 最大的现实价值常常不是推理能力本身，而是可恢复、可追踪、可限额、可审计、可并行和局部上下文隔离。
- 对 Coding Agent，读密集型工作更容易并行；写密集型工作需要更严格的 worktree/文件 ownership/合并策略，否则多个 Agent 会互相制造冲突。

## 目录. 报告结构

- 1 术语澄清：Graph Engineer 到底是什么
  - 2 概念演进：Prompt → Context → Harness → Loop → Graph
  - 3 Loop Engineering：反馈系统的工程化
  - 4 Graph Engineering：执行拓扑的工程化
  - 5 Graph vs Loop：12 个关键维度
  - 6 为什么 Graph  $\neq$  DAG，且 Loop  $\subset$  Graph
  - 7 Graph 的真实收益：并行、上下文、失败隔离、治理
  - 8 Graph 的代价与反模式
  - 9 主流框架证据：LangGraph / AutoGen / Google ADK
  - 10 Coding Agent 场景：为什么“读并行、写谨慎”
  - 11 推荐架构：Graph of Loops
  - 12 从现有长运行 Loop 演进为 Graph Runtime
  - 13 状态模型、节点契约与故障路由
  - 14 评估指标与可观测性
  - 15 POC 路线：4 阶段增量落地
  - 16 最终结论与决策清单
- 附录 A 伪代码与状态 Schema
- 附录 B 来源与参考资料

# 1. 术语澄清：Graph Engineer 到底是什么

这里的 *Graph Engineer* 不是传统 *Knowledge Graph Engineer*。

“Graph Engineer” 在 2026 年 Agent Engineering 讨论中成为一个新的流行标签。LangChain 在 2026-07-22 的官方文章中直接讨论了 “Graph engineering” 这一说法，并指出它虽然带有 buzzword 色彩，但确实描述了真实的设计问题：用图来表达 Agent 系统，可以把工程师对系统行为的先验约束落实为更可控的执行路径，而不是完全依赖 LLM 的临场判断。[S1]

## 1.1 与传统 Knowledge Graph Engineer 的区别

| 维度     | 传统 Knowledge Graph Engineer             | 本文 Graph Engineer                                            |
|--------|-----------------------------------------|--------------------------------------------------------------|
| 工作对象   | 实体、关系、Ontology、RDF/Property Graph       | Agent、Tool、Validator、Human、State、Edge                        |
| 核心数据结构 | 知识图谱                                    | 执行图 / 状态图                                                    |
| 核心问题   | 知识如何结构化、查询、推理                           | 工作如何路由、并行、恢复、验证                                              |
| 典型技术   | Neo4j、RDF、SPARQL、Graph DB               | LangGraph、AutoGen GraphFlow、ADK workflow、custom runtime      |
| 交付物    | Schema / Ontology / Graph Data Pipeline | Workflow topology / runtime / node contracts / observability |

## 1.2 更严谨的工程定义

定义

Graph Engineer = 为 Agentic System 设计 control plane / execution topology 的工程角色。其目标不是让单个 LLM 更聪明，而是让整个系统在复杂任务中以可控、可恢复、可观测的方式协同。

典型职责包括：节点拆分、边和路由条件、共享状态 Schema、局部上下文、fan-out/fan-in、局部循环、checkpoint/resume、权限边界、预算控制、Human-in-the-loop、trace 与 evaluation。

# 2. 概念演进：Prompt → Context → Harness → Loop → Graph

这些词不是版本号，而是不同抽象层。

Agent 工程近年的术语变化，很容易被误读成“新概念淘汰旧概念”。更合理的理解是：随着系统复杂度提高，工程关注点从单次生成逐步上升到上下文、运行环境、反馈回路和全局编排。

| 层级                 | 主要问题       | 典型产物                              |
|--------------------|------------|-----------------------------------|
| Prompt Engineering | 一次调用如何表达任务 | system prompt / few-shot / rubric |

| 层级                  | 主要问题             | 典型产物                                                          |
|---------------------|------------------|---------------------------------------------------------------|
| Context Engineering | 下一步模型真正应该看到什么    | retrieval / memory / compression / isolation                  |
| Harness Engineering | Agent 在什么执行环境里工作 | tools / sandbox / filesystem / permissions / hooks            |
| Loop Engineering    | 如何持续迭代并根据反馈修正    | agent loop / verifier / retry / stop / fresh context          |
| Graph Engineering   | 复杂工作如何组织、路由和治理   | nodes / edges / state / parallelism / subgraphs / checkpoints |

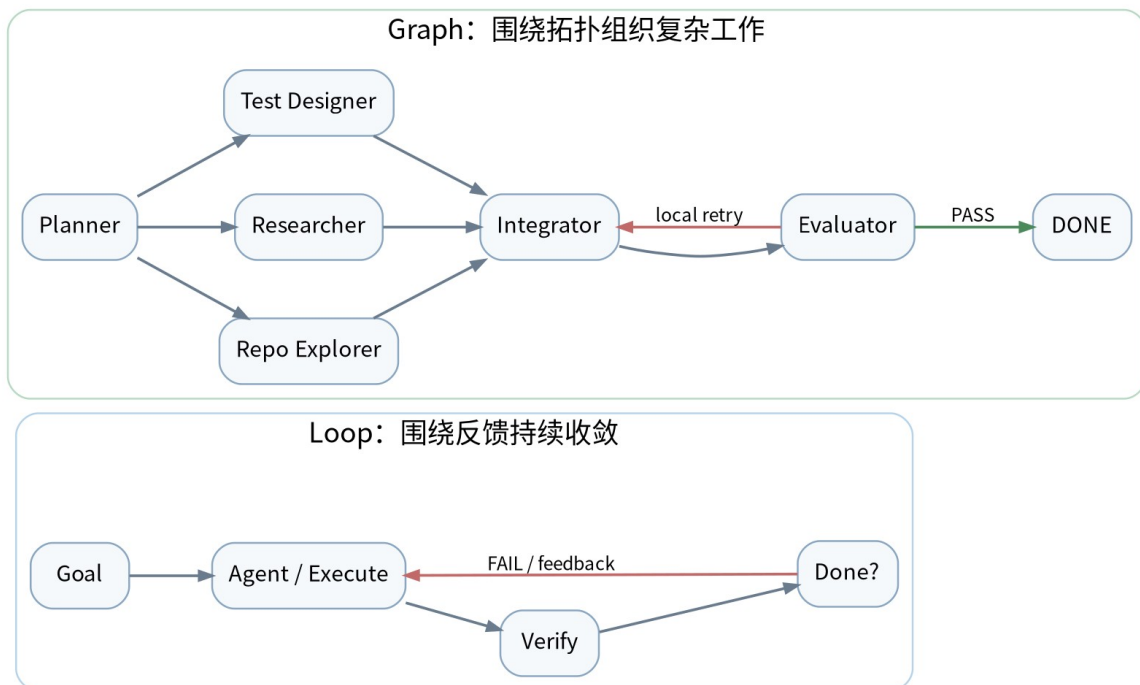
LangGraph 官方目前也清晰区分了 agent framework、agent harness 与 orchestration runtime：LangChain 提供模型/工具/agent loop 抽象，Deep Agents 属于 harness，而 LangGraph 聚焦 durable execution、human-in-the-loop、persistence 等编排能力。[S5]

### 3. Loop Engineering：反馈系统的工程化

*Loop 的核心不是 “while true”，而是可收敛、可终止、可恢复的反馈系统。*

LangChain 对最基本 Agent 的描述是：把上下文给模型，让它在工具调用循环中工作直到完成任务；在此之上，还可以再包一层 verification loop，将输出检查结果作为反馈送回模型。[S2]

#### 3.1 Loop 的最小结构



一个生产级 Loop 至少应显式定义 Goal、Action、Observation、Verifier、Feedback、Memory、Retry budget、Stop condition。缺任何一项都容易退化为“无边界地重复调用模型”。

### 3.2 Loop Engineer 真正优化的指标

- 收敛速度：多少轮达到 acceptance；
- 单轮纠错增益：feedback 是否让下一轮明显更接近目标；
- 失败可解释性：失败是模型、工具、环境、网络还是验收脚本；
- 上下文健康度：是否随轮次膨胀、是否需要 fresh context / summarization；
- 成本与上限：最大轮数、token budget、wall-clock budget；
- 幂等性与恢复：进程崩溃后是否从最近可用状态继续。

### 3.3 Loop 的数学视角

可把 Loop 表达为状态更新： $S(t+1) = F(S(t), \text{Agent}(S(t)), \text{Feedback}(t))$ 。因此 Loop Engineering 的核心问题是“时间维度上的状态如何持续演化，并最终进入可接受的终止集合”。

## 4. Graph Engineering：执行拓扑的工程化

当问题从“一条主线反复修正”变成“多阶段、多角色、多依赖”时，拓扑成为一等对象。

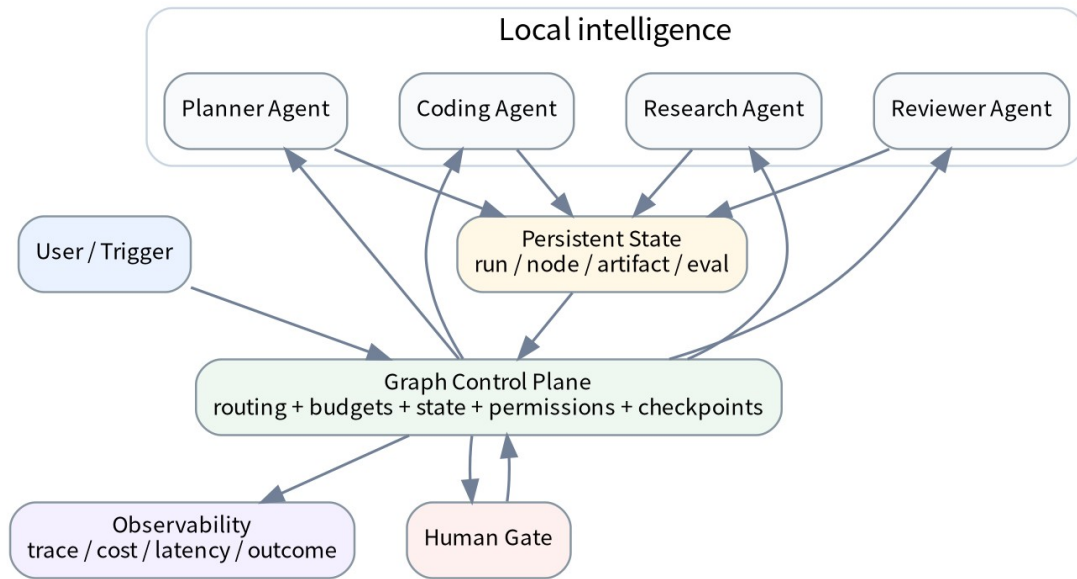
AutoGen 的 GraphFlow 官方文档把这一点说得非常直接：GraphFlow 使用有向图控制 Agent 间执行流，支持 sequential、parallel、conditional 和 looping；当任务需要严格执行顺序、条件分支或复杂循环时，应从自由对话式 team 转向 structured workflow。[S3]

### 4.1 Graph 的四个基本元素

| 元素          | 含义                                      | 工程问题               |
|-------------|-----------------------------------------|--------------------|
| V - Nodes   | Agent / tool / code / validator / human | 这个节点的职责边界和输入输出是什么？ |
| E - Edges   | 允许的状态转移                                 | 什么条件下允许从 A 到 B？    |
| S - State   | 结构化运行状态                                 | 哪些数据是全局的，哪些仅属于节点？  |
| R - Routing | 路由规则/策略                                 | 确定性条件、LLM 决策还是混合？  |

从这个角度看，Graph Engineer 的主要产品不是 Prompt，而是“Execution Topology + State Contract + Runtime Policy”。

## 4.2 全局确定、局部智能



### 推荐原则

Global structure deterministic-ish, local intelligence agentic: 全局结构尽量用代码保证路径、预算和安全边界；局部节点内部再让 Agent 发挥推理与工具使用能力。

## 5. Graph vs Loop: 12 个关键维度

真正的差异不是“单 Agent vs 多 Agent”，而是系统控制面的抽象层级。

| 维度      | Loop Engineer            | Graph Engineer                    |
|---------|--------------------------|-----------------------------------|
| 核心目标    | 持续收敛                     | 全局协调                              |
| 结构      | 反馈环                      | 任意有向状态图                           |
| 并行      | 非核心能力                    | 一等能力                              |
| 分支      | 通常较简单                    | 条件边/动态路由                          |
| 汇合      | 通常隐式                     | fan-in / reducer / integrator     |
| 状态      | 轮次历史、文件状态                | typed graph state + durable store |
| Context | 单 Agent context 管理       | node-local context + handoff      |
| 失败      | 重新迭代                     | fault-domain-specific routing     |
| 恢复      | run/iteration checkpoint | node/super-step checkpoint        |
| 人类介入    | 可插入                      | 可成为显式节点/interrupt                 |
| 可观测性    | iteration trace          | node/edge/state trace             |

| 维度   | Loop Engineer   | Graph Engineer |
|------|-----------------|----------------|
| 主要风险 | 死循环、context rot | 过度编排、协调成本、写冲突  |

## 5.1 两个一句话定义

### Loop Engineer

Feedback-system engineer for autonomous agents: 设计 Goal → Action → Observation → Evaluation → Adaptation 的反馈系统，重点是 Convergence。

### Graph Engineer

Control-plane / topology engineer for agentic systems: 设计 Agents + Loops + Tools + State + Routing + Parallelism + Verification + Human Gates，重点是 Coordination。

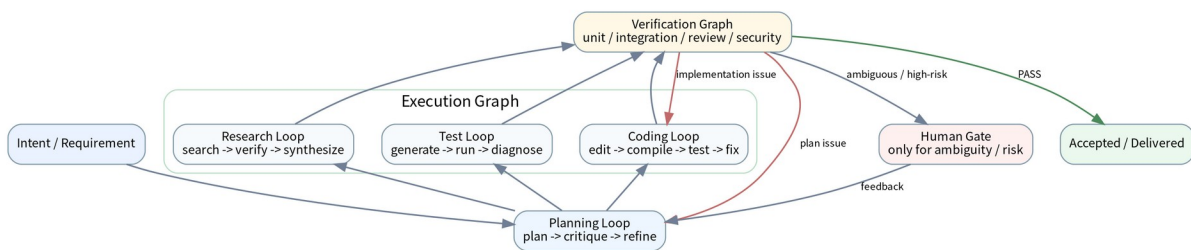
## 6. 为什么 Graph ≠ DAG，且 Loop ⊂ Graph

Agent 工作流通常天然需要回边，因此生产图往往是 *cyclic graph*。

传统数据流水线常被建模成 DAG，但 Agentic Workflow 中至少有四类结构天然产生 cycle：工具调用、验证失败重试、规划修正、Human feedback。LangGraph 早在 2024 年就以 “cyclical graphs needed for agent runtimes” 为主要动机之一；2026 年 Graph Engineering 讨论则进一步把这种结构上升为系统化的工程视角。[S1][S5]

### 6.1 图论关系

最小 Loop 本身可以表示成有向图：A → B → C → A。因此 Loop 是 Graph 的一个子类；Graph 允许多个局部 Loop、不同入口/出口、并行分叉与合流。



成熟 Agent System 更准确的结构是 Graph(Loop1, Loop2, ..., LoopN)，即 “Graph of Loops”。



## 7. Graph 的真实收益

价值集中在四件事：并行、上下文隔离、失败隔离、治理。

### 7.1 并行：缩短关键路径，而不是简单增加 Agent 数

当 Research、Repo analysis、Test design 等子任务相互独立时，Graph 可以 fan-out 并行执行，再 fan-in 汇合。

Google ADK 的 ParallelAgent 就把并发执行 sub-agents 作为内置工作流；AutoGen GraphFlow 同样把 parallel fan-out 作为基础能力。[S3][S4]

关键指标应从“总工作量”转向“critical path latency”。如果三个独立子任务各需要 3 分钟，串行耗时约 9 分钟；理想并行的阶段耗时更接近最慢子任务，而不是三者相加。但实际收益要扣除调度、共享资源争用与汇合成本。

### 7.2 Context isolation: Graph 也是结构化 Context Engineering

多 Agent 的重要价值之一是把探索过程隔离在局部上下文中，只把结构化结果交给上游/下游。LangChain 的动态 subagents 文章明确强调：subagents 可以隔离 context、避免中间工具调用污染主上下文；当规模扩大到数百个子任务时，程序化 orchestration 比逐次 tool-call delegation 更可靠。[S8]

- Planner：只保留 requirement、约束、repo 摘要；
- Researcher：拿研究问题和来源规范，不需要完整代码变更历史；
- Coder：拿 plan、目标文件、接口契约、相关测试；
- Reviewer：拿 diff、验收标准、关键上下文，而不是整个探索轨迹。

### 7.3 Failure isolation: 错误应该回到“正确的责任节点”

| 失败类型                  | 建议路由                          | 不要做什么             |
|-----------------------|-------------------------------|-------------------|
| 编译/单测错误               | Coding Loop                   | 重启整个研究与规划         |
| 架构不成立                 | Planner                       | 让 Coder 无限局部修补    |
| 需求歧义                  | Human Gate / Requirement node | 让 Agent 自行猜测高影响要求 |
| 网络/Provider transient | Tool retry / runtime retry    | 把错误当逻辑失败重新规划      |
| 测试本身错误                | Test Planner / Test owner     | 让实现代码迎合错误测试       |

### 7.4 Durable execution 与 Human-in-the-loop

LangGraph persistence 文档把 checkpoint 定义为线程级 graph state checkpoint，可用于 conversation continuity、human-in-the-loop、time travel 和 fault tolerance；官方 overview 也把“从失败中恢复并长时间运行”列为核心能力。[S5][S6]

## 8. Graph 的代价与反模式

Graph 不是免费的能力升级；它把模型问题转化成分布式系统与工作流问题。

### 8.1 主要代价

- 协调成本：每次 handoff 都可能损失信息；

- 状态复杂度：共享状态、局部状态、持久状态、artifact 状态需要明确生命周期；
- 并行写冲突：多个 Coding Agent 修改相同文件/接口会产生 merge 和语义冲突；
- 调试复杂度：失败可能来自节点本身，也可能来自路由、state reducer、checkpoint 或 race condition；
- 成本放大：多 Agent 可能通过并行和重复验证显著增加 token/compute；
- 过度确定性：把太多判断硬编码成图，可能限制 Agent 对新情况的适应能力。

## 8.2 五个常见反模式

1. “每个角色都拆一个 Agent”：职责并不能因为换成 Agent 就自动变清晰；很多节点应该是 deterministic code。
2. “Supervisor 决定一切”：如果所有 routing 都让一个大模型自由判断，系统其实仍然是 Big Agent Loop，只是外观变成多 Agent。
3. “无限 fan-out”：并行只在子任务独立、合并成本低时有效；写操作尤其容易冲突。
4. “全局共享所有 context”：这会重新制造 context bloat，抵消 Graph 的关键优势。
5. “所有失败都回 Coder”：会让规划错误、测试错误、环境错误被误当成实现问题。

### 反直觉结论

一个设计良好的 Graph 往往包含大量“非 Agent 节点”：schema validation、budget gate、test runner、git operation、artifact merge、permission check、retry policy 都更适合 deterministic implementation。

## 9. 主流框架证据

*Graph Engineering 这个名字很新，但其基础能力已经进入主流 Agent 框架。*

### 9.1 LangGraph：低层 orchestration runtime

LangGraph 官方 overview 把自身定位为面向 long-running、stateful workflows/agents 的低层编排基础设施，强调 deterministic 与 agentic step 混合、persistence、human-in-the-loop、memory 和 observability。[S5]

### 9.2 Microsoft AutoGen GraphFlow

AutoGen GraphFlow 使用 DiGraph 精确控制多 Agent 执行，当前官方文档明确支持 sequential chain、parallel fan-out、conditional branching、带安全退出条件的 loops。文档也提醒该能力仍属于 experimental feature，因此生产落地需关注 API 变化。[S3]

### 9.3 Google ADK workflow agents

Google ADK 将 SequentialAgent、LoopAgent、ParallelAgent 作为 deterministic template workflow：执行顺序由预定义逻辑决定，而非每一步都询问 LLM。这个设计非常能说明 Graph Engineering 的本质——“协调不必全部 Agentic”。[S4]

| 框架        | 图/工作流抽象                     | 循环 | 并行 | 持久化/恢复 | Human Gate |
|-----------|-----------------------------|----|----|--------|------------|
| LangGraph | StateGraph / Functional API | 强  | 强  | 核心能力   | 核心能力       |
| AutoGen   | GraphFlow / DiGraph         | 支持 | 支持 | 依框架组合  | 可组合        |

| 框架         | 图/工作流抽象                                         | 循环 | 并行 | 持久化/恢复           | Human Gate               |
|------------|-------------------------------------------------|----|----|------------------|--------------------------|
| Google ADK | Sequential/Loop/<br>Parallel workflow<br>agents | 内置 | 内置 | Session/State 体系 | 可通过<br>agent/workflow 设计 |

## 10. Coding Agent 场景：为什么“读并行、写谨慎”

不是所有软件工程工作都适合多 Agent 同时执行。

LangChain 对多 Agent 的总结指出：以“读”为主的任务更容易并行；以“写”为主的任务需要处理上下文传递、隐式决策冲突与输出合并。其文章特别提醒，多 Agent research 往往比 coding 更天然适合宽度优先并行。[S7]

### 10.1 可积极并行的 Coding 前置工作

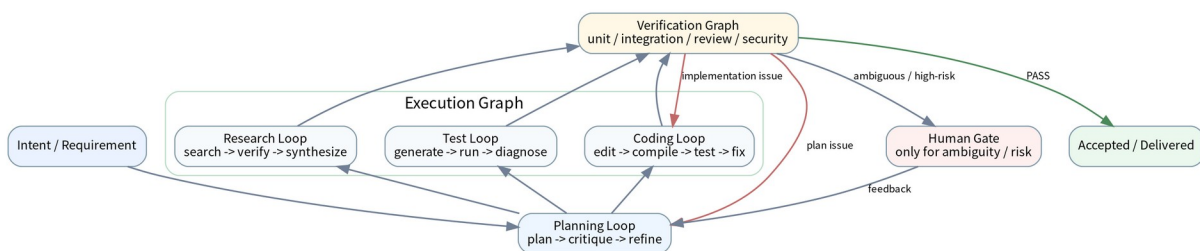
- repo exploration：不同目录/模块独立扫描；
- dependency/API research：外部文档、版本兼容、迁移风险；
- test inventory：已有测试、覆盖缺口、验收案例；
- static review：security、performance、style、architecture 分开检查；
- 日志/故障证据分析：多个假设并行验证。

### 10.2 需要严格约束的并行写

- 通过 worktree / branch / patch 隔离写入；
- 定义 file ownership 或 module ownership，尽量避免多个 worker 修改同一文件；
- 共享接口先冻结 contract，再并行实现；
- fan-in 必须有 Integrator，而不是简单拼接 patch；
- 任何跨模块语义冲突由 integration test + reviewer 重新判断。

## 11. 推荐架构：Graph of Loops

不是把 Loop 删除，而是把它放到正确的层级。



### 11.1 外层 Graph

负责：阶段、依赖、并行、预算、权限、失败路由、checkpoint、人工审批、结果汇合。它是 control plane。

### 11.2 内层 Loop

负责：在局部节点目标内反复行动、观察和纠错。例如 Coding node 内部可以进行 edit → compile → unit test → diagnose → fix；Research node 内部可以 search → verify → fill evidence gap → synthesize。

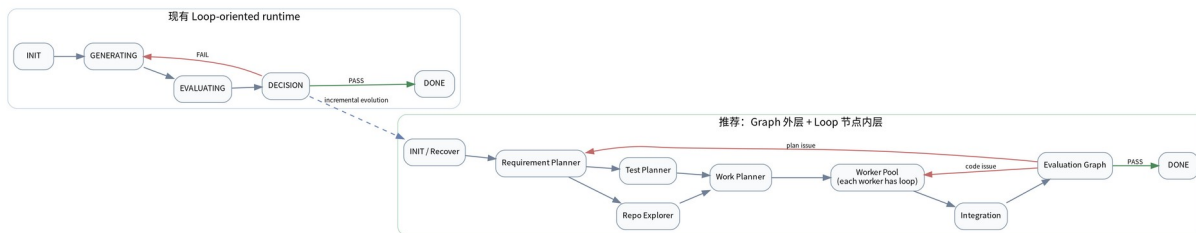
### 11.3 为什么这种组合更稳

- 局部 autonomy 保留模型能力；
- 全局 topology 限制错误扩散；
- 每个 Loop 的 token/step budget 可以独立控制；
- 不同失败可以路由到不同节点，而不是“一切重来”；
- node-level checkpoint 使长运行任务恢复成本显著下降。

## 12. 从现有长运行 Loop 演进为 Graph Runtime

推荐增量改造，不建议推翻重写。

一个典型长运行 Coding Loop 已经拥有 INIT → GENERATING → EVALUATING → DECISION → retry 的 cyclic state machine。它实际上已经具备 Graph 的雏形。最合理的演进是把“整轮 retry”拆成可寻址的节点和故障域，同时保留已有的持久状态、重试、自愈、验收和 Git checkpoint。



### 12.1 哪些现有能力应该原样保留

- crash recovery 与锁释放；
- attempt\_id / run\_id / node\_run\_id；
- acceptance script 与结构化 eval-result；
- Git commit/tag/worktree 作为可回滚 artifact checkpoint；
- provider/network/process error classification；
- wall-clock timeout、model timeout、retry budget。

### 12.2 哪些能力应该新增

| 能力                 | 最小实现                                            | 价值             |
|--------------------|-------------------------------------------------|----------------|
| Node registry      | name + type + input/output schema + permissions | 把阶段变成可寻址执行单元   |
| Conditional edges  | 基于 eval code / failure class 路由                 | 局部修复而不是全局重跑    |
| Fan-out/Fan-in     | 并行只读 worker + deterministic reducer             | 降低关键路径时延       |
| Node-local context | artifact references + minimal prompt context    | 抑制 context rot |
| Node checkpoint    | state snapshot before/after node                | 故障从最近节点恢复      |
| Budget policy      | node/run token/time/step budget                 | 防止图规模失控        |

| 能力          | 最小实现                                       | 价值      |
|-------------|--------------------------------------------|---------|
| Trace model | edge chosen + reason + input/output digest | 可调试、可评估 |

## 13. 状态模型、节点契约与故障路由

*Graph Runtime 的质量很大程度取决于 state contract，而不是 prompt 文案。*

### 13.1 推荐的 State 分层

| 层              | 示例                                       | 生命周期                     |
|----------------|------------------------------------------|--------------------------|
| Run State      | goal, constraints, status, budgets       | 整个任务                     |
| Plan State     | tasks, dependencies, acceptance criteria | 规划版本                     |
| Node State     | node_id, attempt, local_context, result  | 单节点执行                    |
| Artifact State | commit, patch, files, reports, logs      | 跨节点共享的 durable artifacts |
| Eval State     | test_result, review_findings, confidence | 每轮评估                     |
| Runtime State  | provider errors, heartbeat, locks        | 执行基础设施                   |

### 13.2 Node Contract

每个节点至少定义：Input Schema、Output Schema、Allowed Tools、Filesystem Scope、Max Steps、Max Tokens、Timeout、Retry Policy、Checkpoint Policy、Success Predicate、Failure Codes。不要仅靠自然语言“约定”节点行为。

### 13.3 路由优先级

- 6. 先区分基础设施故障与任务逻辑故障。NETWORK / PROVIDER\_TIMEOUT 应优先 runtime retry，不应该触发重新规划。
- 7. 逻辑故障再归因到责任层：implementation → worker，test design → test planner，architecture → planner，requirement ambiguity → human。
- 8. 若连续多轮在同一路径重复失败，应触发 escalation edge，而不是继续原地循环。

## 14. 评估指标与可观测性

*Graph 成功与否必须用系统指标判断，而不是“看起来更复杂”。*

### 14.1 任务层指标

| 指标              | 定义                   | 目的      |
|-----------------|----------------------|---------|
| Acceptance Rate | 最终满足验收的 run 比例       | 总体能力    |
| First-pass Pass | 首次到达 evaluator 即通过比例 | 规划/执行质量 |

| 指标                    | 定义                     | 目的         |
|-----------------------|------------------------|------------|
| Mean Loops to Pass    | 通过前平均局部循环次数            | 收敛效率       |
| Recovery Success      | 故障后从 checkpoint 恢复成功比例 | durability |
| Human Escalation Rate | 需要人工介入的任务比例            | 自动化边界      |

14.2 系统层指标

| 指标                      | 关注点                            |
|-------------------------|--------------------------------|
| Critical-path latency   | 不是总 worker 时长，而是端到端用户等待时间      |
| Token / accepted task   | Graph 是否用更多 token 换来足够质量       |
| Node retry distribution | 哪里是系统真实瓶颈                      |
| Context size per node   | Graph 是否真正实现 context isolation |
| Duplicate work rate     | fan-out 是否重复探索                 |
| Merge conflict rate     | 并行写是否过度                        |
| Edge confusion rate     | 路由错误/反复切换节点                    |
| Checkpoint reuse rate   | 恢复是否真的避免从头重跑                   |

14.3 Trace 必须记录什么

- run\_id / node\_id / node\_attempt;
- model、tools、permission scope;
- input state digest / output state digest;
- edge selected + routing reason;
- token / latency / wall-clock;
- artifact references (commit、patch、log、test report) ;
- failure code 与 retry/escalation decision。

15. POC 路线：4 阶段增量落地

先验证结构性收益，再扩大图的复杂度。

Phase 1：把现有 Loop 显式化为最小 StateGraph

- 保留 INIT / GENERATE / EVALUATE / DECIDE；
- 每个阶段改为 node contract；
- 引入 typed state 与 node-level trace；
- 不增加多 Agent，先验证等价性与恢复能力。

Phase 2：加入“失败路由”，但仍保持单写者

- 区分 implementation / plan / test / environment failure；

- implementation 回 Coder; plan issue 回 Planner;
- 测试环境失败只 runtime retry;
- 衡量平均重跑范围是否缩小。

### Phase 3: 只对读密集型工作做 fan-out

- Repo Explorer、dependency research、test inventory 并行;
- 采用结构化 result schema;
- Integrator 统一压缩为 plan context;
- 比较 latency、token、重复工作率。

### Phase 4: 谨慎引入并行写

- worktree 隔离;
- module/file ownership;
- 契约冻结后再 fan-out;
- Integrator + integration tests 作为强制 fan-in;
- 如果 merge conflict rate 或返工率恶化, 回退为单写者。

#### POC 成功标准

Graph 化必须至少在一个关键维度产生可量化收益: 更高 acceptance、更低端到端 latency、更低恢复成本、更小 context、或更少人工干预。若只是节点更多、trace 更漂亮但成本和质量无改善, 就不应扩大 Graph。

## 16. 最终结论与决策清单

能用 Loop 解决的, 不要为了 Graph 而 Graph; 需要系统级协调时, Graph 才值得。

### 16.1 最终判断

#### 结论 1

Loop Engineer 与 Graph Engineer 不构成“旧职业 / 新职业”的替代关系。Loop 解决反馈收敛, Graph 解决执行拓扑; Graph 必须包含或调用 Loop 才能完成大量现实 Agent 工作。

#### 结论 2

对于长运行 Coding Agent, 最稳妥的演进方向是“Graph 外层 + Loop 内层”: 外层负责阶段、并行、失败域、checkpoint、预算和人工节点, 内层让 Coding/Research/Test Agent 在局部目标中自由迭代。

#### 结论 3

Graph Engineering 的真正护城河不是“画图”, 而是 runtime engineering: durable state、fault routing、context boundary、parallel safety、observability、evaluation 与 governance。

## 16.2 什么时候保持 Loop

- 任务主线清晰、依赖强串行；
- 一个 Agent 能掌握必要上下文；
- 失败通常只需继续修改同一目标；
- 并行子任务很少；
- 任务价值不足以承担多 Agent 成本。

## 16.3 什么时候升级 Graph

- 任务天然包含多个独立阶段/角色；
- 存在明显可并行的读/分析子任务；
- 单上下文已经出现 context rot 或 token bloat；
- 失败类型不同且应该回到不同责任节点；
- 任务长、昂贵，需要 checkpoint/resume；
- 需要 Human approval、权限边界、审计与可观测性；
- 需要针对 node 分别选择不同模型/工具/预算。

## 16.4 最后一句话

**Loop Engineering 让 AI 工程师“持续工作”；Graph Engineering 让一群 AI 工程师“作为一个系统工作”。**

# 附录 A. 伪代码与 State Schema

给出不绑定具体框架的参考接口。

## A.1 Graph of Loops 伪代码

```
state = load_or_create_run(goal)

while not state.done:
    node = router(state)
    checkpoint(state, before=node)

    result = execute_node_loop(
        node=node,
        local_context=build_min_context(state, node),
        budgets=policy.for_node(node),
    )

    state = reduce(state, result)
    checkpoint(state, after=node)

    if result.infrastructure_error:
        runtime_retry(node)
    elif result.logic_failure:
        state.next = failure_router(result.failure_code)
    else:
```



```
state.next = success_edge(node)
```

## A.2 推荐 State Schema (示意)

```
RunState {
  run_id, goal, constraints, status,
  plan_version, current_node, budgets,
  artifacts[], node_runs[], evals[],
  failure_history[], human_feedback[]
}

NodeRun {
  node_id, attempt, input_digest,
  context_refs[], tools[], model,
  started_at, ended_at, tokens,
  result_ref, failure_code, next_edge
}
```

## 附录 B. 来源与参考资料

优先采用框架官方文档与官方技术博客；“Graph Engineer”作为职业名称仍处于术语形成期。

### [S1] LangChain — 3 Years of Graph Engineering with LangGraph (2026-07-22)

Graph engineering 术语、确定性路径与 agentic step 的平衡。 [访问来源](#)

### [S2] LangChain — The Art of Loop Engineering (2026-06-16)

Agent loop、verification loop 与 loop stacking。 [访问来源](#)

### [S3] Microsoft AutoGen — GraphFlow (Workflows) (accessed 2026-08)

DiGraph、sequential/parallel/conditional/looping，GraphFlow 当前为 experimental。 [访问来源](#)

### [S4] Google Agent Development Kit — Template workflow agents (accessed 2026-08)

SequentialAgent、LoopAgent、ParallelAgent 的 deterministic workflow。 [访问来源](#)

### [S5] LangGraph Official Docs — Overview (accessed 2026-08)

durable execution、deterministic + agentic、human-in-the-loop、memory、observability。 [访问来源](#)

### [S6] LangGraph Official Docs — Persistence (accessed 2026-08)

checkpointer、store、resume、fault tolerance、thread-scoped state。 [访问来源](#)

### [S7] LangChain — How and when to build multi-agent systems (2025-06-16)

context engineering、多 Agent 读/写任务的并行边界、durable execution。 [访问来源](#)

### [S8] LangChain — Introducing Dynamic Subagents in Deep Agents (2026-06-29)

context isolation、大规模 fan-out、程序化 orchestration。 [访问来源](#)

研究说明：本文将“Graph Engineer”视为 Agentic Engineering 圈正在形成的工程术语，而非已经标准化的人力资源岗位名称。文中关于架构适用性、迁移路线和设计原则的部分，是在官方框架能力与多 Agent 公开工程经验之上的综合判断。